

1. Import the attached Netflix csv file in Jupyter notebook and perform following operations using Pandas:

Step 1: Import necessary libraries

In [117...

```
import pandas as pd
```

Step 2: Load the dataset (replace 'netflix_titles.csv' with the actual path to your file)

In [118...

```
df = pd.read_excel('C:\\Users\\user\\Documents\\Netflix.xlsx')  
df
```

Out[118...

	show_id	type	title	director	cast	country	date_added	release_y
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2

	show_id	type	title	director	cast	country	date_added	release_y
	4	s5 TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2
	...	...	...	...	...	...	...	
	8775	s8803 Movie	Zodiac	David Fincher	Mark Ruffalo, Jake Gyllenhaal, Robert Downey J...	United States	November 20, 2019	2
	8776	s8804 TV Show	Zombie Dumb	NaN	NaN	NaN	July 1, 2019	2
	8777	s8805 Movie	Zombieland	Ruben Fleischer	Jesse Eisenberg, Woody	United States	November 1, 2019	2

	show_id	type	title	director	cast	country	date_added	release_y
					Harrelson, Emma Stone, ...			
8778	s8806	Movie	Zoom	Peter Hewitt	Tim Allen, Courteney Cox, Chevy Chase, Kate Ma...	United States	January 11, 2020	2020
8779	s8807	Movie	Zubaan	Mozez Singh	Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...	India	March 2, 2019	2019

8780 rows × 11 columns

a) Print the first 5 rows and last 5 rows of the dataframe

In [119...

```
print("First 5 rows:")  
n=df.head()  
n
```

First 5 rows:

Out[119...

	show_id	type	title	director	cast	country	date_added	release_year
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021

	show_id	type	title	director	cast	country	date_added	release_year
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021

In [120...

```
print("Last 5 rows:")
r=df.tail()
r
```

Last 5 rows:

Out[120...

	show_id	type	title	director	cast	country	date_added	release_y
8775	s8803	Movie	Zodiac	David Fincher	Mark Ruffalo, Jake Gyllenhaal, Robert Downey J...	United States	November 20, 2019	2
8776	s8804	TV Show	Zombie Dumb	NaN	NaN	NaN	July 1, 2019	2
8777	s8805	Movie	Zombieland	Ruben Fleischer	Jesse Eisenberg, Woody Harrelson, Emma Stone, ...	United States	November 1, 2019	2
8778	s8806	Movie	Zoom	Peter Hewitt	Tim Allen, Courteney Cox, Chevy Chase	United States	January 11, 2020	2

	show_id	type	title	director	cast	country	date_added	release_y
					Chase, Kate Ma...			
8779	s8807	Movie	Zubaan	Mozez Singh	Vicky Kaushal, Sarah- Jane Dias, Raaghav Chanan...	India	March 2, 2019	2

b) Check how many rows and columns are there using Pandas function

In [121...

```
df.shape
```

Out[121...

```
(8780, 11)
```

c) Print all the column names

In [122...

```
print("Column names:", df.columns)
```

```
Column names: Index(['show_id', 'type', 'title', 'director', 'cast', 'country',  
                    'date_added',  
                    'release_year', 'rating', 'duration', 'listed_in'],  
                    dtype='object')
```

d) Calculate the descriptive statistics of all variables (integer/float/object, etc.)

In [123...

```
df.describe(include='all')
```

Out[123...

	show_id	type	title	director	cast	country	date_added	release
count	8780	8780	8780	6150	7956	7952	8770	8780.00
unique	8780	2	8780	4516	7671	745	1765	
top	s1	Movie	Dick Johnson Is Dead	Rajiv Chilaka	David Attenborough	United States	January 1, 2020	
freq	1	6110	1	19	19	2809	108	
mean	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2014.17
std	NaN	NaN	NaN	NaN	NaN	NaN	NaN	8.82
min	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1925.00
25%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2013.00
50%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2017.00
75%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2019.00
max	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2021.00

e) Check the number of unique values for each column

```
In [124... print("Number of unique values in each column:")
p=df.nunique()
p
```

Number of unique values in each column:

```
Out[124... show_id      8780
type           2
title          8780
director       4516
cast           7671
country        745
date_added     1765
release_year   74
rating         14
duration       220
listed_in      514
dtype: int64
```

f) Check the percentage of missing values for each column

```
In [125... print("\nPercentage of missing values in each column:")
missing_percent = (df.isnull().sum() / len(df)) * 100
```

```
print(missing_percent)
```

Percentage of missing values in each column:

show_id	0.000000
type	0.000000
title	0.000000
director	29.954442
cast	9.384966
country	9.430524
date_added	0.113895
release_year	0.000000
rating	0.045558
duration	0.000000
listed_in	0.000000

dtype: float64

g) Delete all the rows where the Director column has missing values

In [126...

```
df1 = df.dropna(subset=['director'])
```

h) Print all the records where 'country' has Germany value (including West Germany)

```
In [137... germany_filter = df['country'].str.contains('Germany', case=False, na=False)
df_germany = df[germany_filter]
print("Records where 'country' contains 'Germany' or 'West Germany':")
df_germany.head()
```

Records where 'country' contains 'Germany' or 'West Germany':

Out[137...

	show_id	type	title	director	cast	country	date_added	relea
7	s8	Movie	Sankofa	Haile Gerima	Kofi Ghanaba, Oyafunmike Ogunlano, Alexandra D...	United States, Ghana, Burkina Faso, United Kin...	September 24, 2021	
12	s13	Movie	Je Suis Karl	Christian Schwochow	Luna Wedler, Jannis Niewöhner, Milan Peschel,...	Germany, Czech Republic	September 23, 2021	
129	s130	Movie	An Unfinished Life	Lasse Hallström	Robert Redford, Jennifer Lopez, Morgan Freeman...	Germany, United States	September 1, 2021	
142	s143	Movie	Freedom Writers	Richard LaGravenese	Hilary Swank, Patrick	Germany, United States	September 1, 2021	

show_id	type		title	director	cast	country	date_added	relea
					Dempsey, Scott Glenn, Im...			
172	s173	Movie	School of Rock	Richard Linklater	Jack Black, Joan Cusack, Mike White, Sarah Sil...	United States, Germany	September 1, 2021	

i) Expand Duration column into 2 separate columns – numeric value and string

In [128...

```
df[['duration_value', 'duration_unit']] = df['duration'].str.extract(r'(\d+)\s*(
print("\nDuration split into two columns:")
print(df[['duration', 'duration_value', 'duration_unit']].head())
```


Duration split into two columns:

	duration	duration_value	duration_unit
0	90 min	90	min
1	2 Seasons	2	Seasons
2	1 Season	1	Season
3	1 Season	1	Season
4	2 Seasons	2	Seasons

j) Split Date added into 3 separate columns (day, month, year)

In [135...

```
df['date'] = pd.to_datetime(df['date_added'], errors='coerce')
df['day'] = df['date'].dt.day
df['month'] = df['date'].dt.month
df['year'] = df['date'].dt.year
t=df[['day', 'month', 'year']].head()
t
```

Out[135...

	day	month	year
0	25.0	9.0	2021.0
1	24.0	9.0	2021.0
2	24.0	9.0	2021.0
3	24.0	9.0	2021.0
4	24.0	9.0	2021.0

k) Print the number of TV shows/Movies released in each year

In [132...

```
print("Number of TV shows/Movies released each year:")
i=df['release_year'].value_counts().sort_index()
i
```

Number of TV shows/Movies released each year:

```
Out[132... release_year
1925      1
1942      2
1943      3
1944      3
1945      4
...
2017    1030
2018    1141
2019    1024
2020     952
2021     592
Name: count, Length: 74, dtype: int64
```

I) Rename the column 'title' to 'movie_title'

```
In [20]: df.rename(columns={'title': 'movie_title'}, inplace=True)
print("\nRenamed column 'title' to 'movie_title':")
print(df.columns)
```

```
Renamed column 'title' to 'movie_title':
Index(['show_id', 'type', 'movie_title', 'director', 'cast', 'country',
      'date_added', 'release_year', 'rating', 'duration', 'listed_in',
      'duration_value', 'duration_unit', 'day_added', 'month_added',
      'year_added'],
      dtype='object')
```


2. Import both the attached files (student.csv and marks.csv) in Jupyter notebook and perform following operations:

a) Combine both the dataframes into single dataframe which will have all the records from both the tables.

```
In [1]: import pandas as pd

sdf = pd.read_csv('C:\\Users\\user\\Downloads\\student.csv')
mdf = pd.read_csv('C:\\Users\\user\\Documents\\markes.csv')
```

```
In [2]: cdf = pd.merge(sdf, mdf, on='Student_id', how='outer')
cdf
```

Out[2]:

	Student_id	Age	Gender	Grade	Employed	Mark	City
0	1	19	Male	1st Class	yes	95.0	Chennai
1	2	20	Female	2nd Class	no	70.0	Delhi
2	3	18	Male	1st Class	no	98.0	Mumbai
3	4	21	Female	2nd Class	no	75.0	Pune
4	5	19	Male	1st Class	no	89.0	Kochi
...	...	...	...	...	...	...	...
227	228	21	Female	1st Class	no	99.0	Pune
228	229	20	Male	2nd Class	no	70.0	Chennai
229	230	20	Male	3rd Class	yes	55.0	Delhi
230	231	19	Female	1st Class	yes	97.0	Mumbai
231	232	20	Male	3rd Class	yes	59.0	Pune

232 rows × 7 columns

b) Print the maximum and minimum marks Gender wise.

```
In [3]: max_marks = cdf.groupby('Gender')['Mark'].max()
min_marks = cdf.groupby('Gender')['Mark'].min()

print("Maximum Marks Gender-wise:\n", max_marks)
print("Minimum Marks Gender-wise:\n", min_marks)
```

Maximum Marks Gender-wise:

Gender

Female 99.0

Male 100.0

Name: Mark, dtype: float64

Minimum Marks Gender-wise:

Gender

Female 40.0

Male 40.0

Name: Mark, dtype: float64

c) Print all the students IDs and their marks who have scored more than the average marks of the class.

```
In [4]: avg = cdf['Mark'].mean()
```

```
above_avg = cdf[cdf['Mark'] > avg][['Student_id', 'Mark']]
above_avg
```

Out[4]:

	Student_id	Mark
0	1	95.0
2	3	98.0
3	4	75.0
4	5	89.0
11	12	90.0
...	...	...
221	222	74.0
222	223	79.0
224	225	72.0
227	228	99.0
230	231	97.0

137 rows × 2 columns

d) Print the dataframe who are Males and are Employed.

```
In [5]: # Print the dataframe for males who are employed
empM = cdf[(cdf['Gender'] == 'Male') & (cdf['Employed'] == 'yes')]
empM
```

Out[5]:

	Student_id	Age	Gender	Grade	Employed	Mark	City
0	1	19	Male	1st Class	yes	95.0	Chennai
5	6	20	Male	2nd Class	yes	69.0	Gwalior
7	8	21	Male	3rd Class	yes	54.0	Chennai
12	13	19	Male	1st Class	yes	92.0	Gwalior
14	15	19	Male	1st Class	yes	97.0	Pune
16	17	20	Male	2nd Class	yes	68.0	Gwalior
18	19	21	Male	2nd Class	yes	72.0	Chennai
29	30	19	Male	1st Class	yes	81.0	Chennai
34	35	20	Male	2nd Class	yes	79.0	Gwalior
36	37	21	Male	3rd Class	yes	53.0	Pune
41	42	19	Male	1st Class	yes	86.0	Delhi
43	44	19	Male	1st Class	yes	95.0	Pune
45	46	20	Male	2nd Class	yes	79.0	Delhi
47	48	21	Male	2nd Class	yes	67.0	Pune

	Student_id	Age	Gender	Grade	Employed	Mark	City
58	59	19	Male	1st Class	yes	89.0	Pune
63	64	20	Male	2nd Class	yes	78.0	Delhi
65	66	21	Male	3rd Class	yes	59.0	Pune
70	71	19	Male	1st Class	yes	85.0	Kochi
72	73	19	Male	1st Class	yes	89.0	Bhopal
74	75	20	Male	2nd Class	yes	79.0	Delhi
76	77	21	Male	2nd Class	yes	77.0	Pune
87	88	19	Male	1st Class	yes	96.0	Pune
92	93	20	Male	2nd Class	yes	74.0	Kochi
94	95	21	Male	3rd Class	yes	50.0	Bhopal
99	100	19	Male	1st Class	yes	84.0	Kochi
101	102	19	Male	1st Class	yes	98.0	Chennai
103	104	20	Male	2nd Class	yes	78.0	Mumbai
105	106	21	Male	2nd Class	yes	77.0	Kochi

	Student_id	Age	Gender	Grade	Employed	Mark	City
116	117	19	Male	1st Class	yes	90.0	Kochi
121	122	20	Male	2nd Class	yes	75.0	Mumbai
123	124	21	Male	3rd Class	yes	49.0	Chennai
128	129	19	Male	1st Class	yes	89.0	Gwalior
130	131	19	Male	1st Class	yes	90.0	Chennai
132	133	20	Male	2nd Class	yes	75.0	Mumbai
134	135	21	Male	2nd Class	yes	75.0	Kochi
145	146	19	Male	1st Class	yes	98.0	Mumbai
150	151	20	Male	2nd Class	yes	78.0	Pune
152	153	21	Male	3rd Class	yes	46.0	Gwalior
157	158	19	Male	1st Class	yes	92.0	Pune
159	160	19	Male	1st Class	yes	92.0	Delhi
161	162	20	Male	2nd Class	yes	79.0	Pune
163	164	21	Male	2nd Class	yes	75.0	Gwalior

	Student_id	Age	Gender	Grade	Employed	Mark	City
174	175	19	Male	1st Class	yes	84.0	Pune
179	180	20	Male	2nd Class	yes	76.0	Delhi
181	182	21	Male	3rd Class	yes	58.0	Pune
186	187	19	Male	1st Class	yes	83.0	Kochi
188	189	19	Male	1st Class	yes	89.0	Bhopal
190	191	20	Male	2nd Class	yes	77.0	Delhi
192	193	21	Male	2nd Class	yes	75.0	Pune
203	204	19	Male	1st Class	yes	91.0	Pune
208	209	20	Male	2nd Class	yes	70.0	Mumbai
210	211	21	Male	3rd Class	yes	58.0	Kochi
215	216	19	Male	1st Class	yes	95.0	Mumbai
217	218	19	Male	1st Class	yes	96.0	Kochi
219	220	20	Male	2nd Class	yes	75.0	Bhopal
221	222	21	Male	2nd Class	yes	74.0	Kochi

	Student_id	Age	Gender	Grade	Employed	Mark	City
229	230	20	Male	3rd Class	yes	55.0	Delhi
231	232	20	Male	3rd Class	yes	59.0	Pune

e) Create a new Column 'IQ_level' which will have 3 values (Intelligent, Mediocre, weak). If student scored than 80 then Tag him as Intelligent, if student scored between 50-80, then Mediocre, else weak.

```
In [6]: def iq(Mark):
        if Mark > 80:
            return 'Intelligent'
        elif Mark >= 50:
            return 'Mediocre'
        else:
            return 'Weak'

cdf['IQ_level'] = cdf['Mark'].apply(iq)

cdf
```

Out[6]:

	Student_id	Age	Gender	Grade	Employed	Mark	City	IQ_level
0	1	19	Male	1st Class	yes	95.0	Chennai	Intelligent
1	2	20	Female	2nd Class	no	70.0	Delhi	Mediocre
2	3	18	Male	1st Class	no	98.0	Mumbai	Intelligent
3	4	21	Female	2nd Class	no	75.0	Pune	Mediocre
4	5	19	Male	1st Class	no	89.0	Kochi	Intelligent
...	...	...	...	...	...	...	...	...
227	228	21	Female	1st Class	no	99.0	Pune	Intelligent
228	229	20	Male	2nd Class	no	70.0	Chennai	Mediocre
229	230	20	Male	3rd Class	yes	55.0	Delhi	Mediocre
230	231	19	Female	1st Class	yes	97.0	Mumbai	Intelligent
231	232	20	Male	3rd Class	yes	59.0	Pune	Mediocre

232 rows × 8 columns

f) Count the number of males and females from each city.

```
In [7]: gender_count = cdf.groupby(['City', 'Gender']).size().unstack(fill_value=0)
gender_count
```

Out[7]:

Gender	Female	Male
--------	--------	------

City		
Bhopal	13	15
Chennai	15	18
Delhi	12	20
Gwalior	14	18
Kochi	10	21
Mumbai	16	16
Pune	15	26

g) Print the top 5 Male scorers


```
In [8]: scorers = cdf[cdf['Gender'] == 'Male'].nlargest(5, 'Mark')
print("Top 5 Male Scorers:\n", scorers)
```

Top 5 Male Scorers:

	Student_id	Age	Gender	Grade	Employed	Mark	City	IQ_level
149	150	19	Male	1st Class	no	100.0	Bhopal	Intelligent
147	148	18	Male	1st Class	no	99.0	Kochi	Intelligent
2	3	18	Male	1st Class	no	98.0	Mumbai	Intelligent
60	61	18	Male	1st Class	no	98.0	Gwalior	Intelligent
62	63	19	Male	1st Class	no	98.0	Chennai	Intelligent

h) Replace the Male value with M and Female value with F and export this dataframe to excel file in D: (D drive) and name the file as test.csv.

```
In [15]: import os

cdf['Gender'] = cdf['Gender'].replace({'Male': 'M', 'Female': 'F'})

directory = 'C:\\Users\\user\\Desktop'
file_name = 'test.csv'

file_path = os.path.join(directory, file_name)

cdf.to_csv(file_path, index=False)
```

```
print("=== Modified DataFrame Table ===")  
print(f"CSV saved at: {file_path}")  
cdf
```

```
=== Modified DataFrame Table ===  
CSV saved at: C:\Users\user\Desktop\test.csv
```

Out[15]:

	Student_id	Age	Gender	Grade	Employed	Mark	City	IQ_level
0	1	19	M	1st Class	yes	95.0	Chennai	Intelligent
1	2	20	F	2nd Class	no	70.0	Delhi	Mediocre
2	3	18	M	1st Class	no	98.0	Mumbai	Intelligent
3	4	21	F	2nd Class	no	75.0	Pune	Mediocre
4	5	19	M	1st Class	no	89.0	Kochi	Intelligent
...	...	...	...	...	...	...	...	...
227	228	21	F	1st Class	no	99.0	Pune	Intelligent
228	229	20	M	2nd Class	no	70.0	Chennai	Mediocre
229	230	20	M	3rd Class	yes	55.0	Delhi	Mediocre
230	231	19	F	1st Class	yes	97.0	Mumbai	Intelligent
231	232	20	M	3rd Class	yes	59.0	Pune	Mediocre

232 rows × 8 columns

i) Check if any student_ID is duplicated.

```
In [11]: Duplicates = cdf[cdf.duplicated(['Student_id'], keep='last')]
         Duplicates
```

```
Out[11]:  Student_id  Age  Gender  Grade  Employed  Mark  City  IQ_level
```

j) Create a separate dataframe which will have all the Integer/Float variables.

```
In [16]: numdf = cdf.select_dtypes(include=['int', 'float'])
         print("DataFrame with Integer/Float Variables")
         numdf
```

DataFrame with Integer/Float Variables:

Out[16]:

	Student_id	Age	Mark
0	1	19	95.0
1	2	20	70.0
2	3	18	98.0
3	4	21	75.0
4	5	19	89.0
...	...	...	...
227	228	21	99.0
228	229	20	70.0
229	230	20	55.0
230	231	19	97.0
231	232	20	59.0

232 rows × 3 columns

k) Get those Student_IDs which are present in Students table but not in Marks table.

```
In [18]: missing_ids = sdf[~sdf['Student_id'].isin(mdf['Student_id'])]  
print("Student IDs in Students table but not in Marks table:")  
missing_ids
```

Student IDs in Students table but not in Marks table:

```
Out[18]:
```

	Student_id	Age	Gender	Grade	Employed
8	9	22	Female	3rd Class	yes
9	10	21	Male	1st Class	no
48	49	19	Male	2nd Class	no

3. Explain the concept of missing values.
How can you identify the missing values in a Pandas DataFrame ?

What are the different ways of treating/Imputing/Deleting the missing values.

Explain with example.

Concept of Missing Values

Missing values in a dataset represent the absence of a value for a particular feature in a specific observation. These can occur for various reasons, such as data entry errors, equipment malfunctions, or respondents not answering survey questions. Missing values can lead to biased analyses, reduce statistical power, and complicate modeling, making it essential to address them appropriately.

Identifying Missing Values in a Pandas DataFrame

In a Pandas DataFrame, missing values can be identified using several methods:

1. **Using the `isnull()` method:** This method returns a DataFrame of the same shape as the original, containing `True` for missing values and `False` for non-missing values.
2. **Using the `sum()` method:** By applying `isnull()` followed by `sum()`, you can get a count of missing values in each column.
3. **Using the `info()` method:** This method provides a concise summary of the DataFrame, including the count of non-null values for each column, which can help identify columns with missing values.

Treating/Imputing/Deleting Missing Values

There are several strategies to handle missing values, including:

1. **Deletion:**

- **Listwise Deletion:** This involves removing any row that contains at least one missing value. While simple, it can result in a significant loss of data, especially if many rows contain missing values.
- **Pairwise Deletion:** This approach only excludes missing values for specific analyses rather than entire rows, preserving more data but complicating the

analysis.

2. **Imputation:**

- **Mean/Median/Mode Imputation:** For numerical columns, replacing missing values with the mean or median is common. For categorical columns, the mode can be used. This method is easy but can reduce variability and introduce bias.
- **Forward/Backward Fill:** In time series data, forward filling involves replacing missing values with the last observed value, while backward filling uses the next observed value. This method is useful for preserving temporal continuity.
- **Interpolation:** This technique estimates missing values based on existing data points. Linear interpolation uses the average of neighboring values, while polynomial interpolation can model more complex patterns.
- **Using Algorithms:** Advanced techniques involve using algorithms like k-Nearest Neighbors (k-NN) or machine learning models to predict and fill missing values based on other features in the dataset.

Example of Handling Missing Values

Suppose you have a dataset containing information about students, including their scores in mathematics, science, and English. If some entries for mathematics scores are missing, you might decide to handle them in the following ways:

- **Deletion:** If the number of students with missing math scores is small, you may opt to delete those rows to avoid bias in your analysis.
- **Mean Imputation:** You could calculate the mean score of the mathematics scores from the existing data and replace the missing values with this mean.
- **Forward Fill:** If the dataset is sorted by student ID or date of entry, and a student missed a score due to a recent absence, you might replace the missing score with the last valid score from the previous entry.

By understanding and appropriately handling missing values, you can ensure more reliable and accurate analysis in your data-driven projects.